

ClaudeForth 6809 — Session Summary, Part 2

Scope: continues directly from the previous summary's end point (double-number text input flagged as `[PENDING]` , not yet implemented) through the end of an extensive MAME-based manual testing and bug-fixing phase, a full documentation resync, and a memory-map diagram rebuild — stopping just before new assembly-level unit tests were added (a separate, later phase not covered here).

Written for transfer to a parallel conversation building a 68008 ANS Forth with equivalent capabilities. Where a finding is CPU-specific (6809 quirks), it's flagged explicitly as “verify your own CPU's equivalent” rather than something to port directly — the *pattern* of catching it is the transferable part, not the specific instruction.

1. ANS Forth semantics clarified this phase (CPU-independent — directly transferable)

These are correctness facts about the ANS standard itself, confirmed by reading the spec text precisely (not from memory), independent of any particular CPU implementation:

- **Double-number text input (3.4.1.3):** a number immediately followed by a decimal point (`123.`) converts to a double-cell number instead of single-cell (`1234 0` for `1234.`). Implemented by: detecting the trailing `.` before the digit-parsing loop runs (excluding it from the digit count), performing full double-cell negation for the negative case, and re-deriving the double-vs-single decision at the very end by re-reading the original parsed text rather than carrying a flag through the whole routine (useful when you're tight on scratch registers/RAM and the original text is still available unchanged).
- **CATCH / THROW 's actual, precise guarantee** (confirmed by reading the DPANS THROW glossary text directly, not inferred): on a caught exception, **only the stack depth is guaranteed restored, not the values**. The `i*x` stack arguments “could have been modified arbitrarily during the execution of xt” — an implementation is free to leave *any* garbage in that position; different implementations (confirmed by comparing against gforth) legitimately leave *different* garbage there. The standard's own guidance: applications should `DROP` those items rather than try to interpret them. Don't assume `CATCH` “rolls back” the original arguments — it doesn't, by design.
- **ENVIRONMENT? 's unknown-attribute behavior:** if the query string isn't recognized, the

result is plain `false` (one stack item), not `(0 true)` (two items) — these are genuinely different, and conflating them is a real bug class (see §2 below). An unimplemented *optional word set's* queries (e.g. `WORDLISTS` when Search-Order isn't implemented) should correctly fall into "unrecognized," not report a fabricated value.

- **SIGN 's usage pattern:** a very common mistake (caught live during this session) is calling `SIGN` *after* `#S` has already fully consumed the `ud` down to `0 0` — at that point `SIGN` sees `0` (not negative) regardless of the original number's sign, since the true signed value never reaches the top of the stack in that sequence. The standard idiom stashes the sign on the return stack before conversion and restores it right before `SIGN`: `DUP >R ABS S>D <# #S R> SIGN #> .`
- **HOLD -family words build pictured output right-to-left:** whatever is `HOLD` ed *first* ends up *rightmost* in the final string, since each `HOLD` prepends to what's already been built. A prefix label added before the digits (`HOLDS` called before `#S`) ends up as a *suffix* in the output — this is correct, standard behavior, not a bug, but it trips people up. Fix is word-order (`#S` before the label's `HOLDS`), not a code change.
- **WORDLISTS and similar "size of a still-unimplemented feature" queries** correctly report `false` , not a fabricated capacity number — don't invent a plausible-looking value just because the query is supported syntactically.

2. Real bugs found and fixed this phase

Two bug *classes* dominate this list. Both are CPU-specific to the 6809 — **flagged explicitly below as things to check against your own CPU's actual documented behavior, not assumptions to carry over.**

Class A: `PULU` / `PULS` do not affect condition codes on genuine 6809

This is a real, documented 6809 quirk: `pop` instructions are pure data movement, they don't set N/Z/C the way `LDA` / `LDD` do. Code that does `PULU D` immediately followed by a branch (`BPL` , `BEQ` , etc.) that *assumes* the branch tests the just-popped value is testing **stale flags from whatever ran before the pop** instead. This bug was found independently in at least four different places this session (`TRYNUM` , `STOD` / `S>D` , `SIGN` , and implicitly informed how `NUMBERQ` 's own negation logic was designed) — it's a genuine, recurring pitfall, not a one-off typo.

Fix pattern: after any `PULU` / `PULS` , explicitly re-test the popped value (`CMPD #0` , `TSTA` , etc.) before branching on it — never assume the flags reflect the pop itself.

A subtlety found while fixing this: some *callers* of a buggy routine were accidentally, fragile-luck protected — e.g. `SIGN` 's four existing internal callers (`. , .R , D. , D.R`) each happened to run a flag-setting `LDD` of the true signed value immediately before calling `SIGN` , masking `SIGN` 's own bug for those specific call sites. A *direct*, standalone call to `SIGN` (no such lucky preceding instruction) failed outright. Lesson: “the existing callers work” is not the same as “the routine is correct” — test the routine in isolation, not just through its accidentally-protective callers.

Action for 68008 (or any CPU): check your own architecture's documented flag-setting behavior for its stack-pop/move instructions (`MOVE (A7)+, D0` and similar on 68008) — do NOT assume they set flags just because it “feels like” a data-movement-with-side-effect operation should. Check the actual programmer's reference, not intuition.

Class B: 6809's `COM` (one's complement) unconditionally sets carry, regardless of operand

Discovered while implementing 32-bit negation for double-number input. The negation algorithm (complement both cells, add 1, propagate carry into the high cell only if the low-cell addition actually overflowed) had two bugs, found across two separate turns:

1. **The carry-corruption bug:** the code computed the real carry from the low cell's `ADDD #1` , then ran `COM` on the high cell *before* checking that carry — and `COM` unconditionally sets carry=1 regardless of its own operand, silently destroying the real carry before the intended conditional check could read it. Result: the high-cell increment ran *unconditionally* instead of only when the low cell genuinely overflowed — wrong in general, though it happened to look right for some specific test values, which is exactly why it shipped once before being caught by more thorough testing.

Fix: save the true carry (`PSHS CC`) immediately after the carry-producing addition, before any instruction that might disturb it, then restore it (`PULS CC`) right before the branch that depends on it.

2. **A second, previously-masked bug, only exposed by fixing the first:** once the carry check was genuinely working, the branch it guarded turned out to skip *past the store instruction itself*, not just the conditional increment — meaning the complemented high-cell value was computed in a register but never actually written back to memory in the no-carry case. This had been completely hidden by bug #1, since bug #1 meant the branch was *never actually taken* in practice (carry always read as set), so the store always ran (with the wrong value, but it ran).

Lesson, general and CPU-independent: fixing one bug can expose a second, previously-latent one that was masked by the first — don't assume a fix is complete just because the *reported* symptom is gone; re-verify the full logic path, especially anything

the first bug might have prevented from ever executing.

Action for 68008: check whether any 68008 instruction you're relying on for multi-precision arithmetic (complement, negate, shift) has a similarly "surprising, unconditional" flag side effect documented in the 68000 family programmer's reference. The 68000 family is generally more regular about condition codes than the 6809, but verify rather than assume — and specifically re-examine any carry-propagation code for the exact "does an intervening instruction between the carry-producing op and the carry-testing branch disturb it" pattern above.

Other real bugs found (implementation-specific, but the *category* of bug is worth knowing)

- **A stray, uncollected return value left on the stack:** (the comment word) called a `WORD`-equivalent routine to skip to the closing delimiter, but never consumed the address that routine always returns — leaving a stray value on the stack after every comment, and a compile-time stack-depth-check mismatch for any colon definition containing one. **General lesson:** when a helper routine has a fixed, unconditional return-value contract, every caller must account for *all* of it, even parts the caller doesn't need — either consume and discard, or don't call a routine whose contract doesn't match your needs.
- **A table-walking routine's register clobbered by a helper it calls:** `ENVIRONMENT?`'s dispatcher used a register as a walk-the-table pointer across a loop, but a string-comparison helper it called used that same register as its own internal scratch, without the caller saving/restoring it. After a match, the "read the matched entry's value" step read from wherever the *helper* had left the register — which, for one specific query, happened to land exactly at the start of the *next* table entry's string data, producing a precisely explainable (not just "garbage") wrong value. **General lesson:** any register/variable used across a call to another routine needs an explicit contract about who preserves it — "it happened to work before" is not a contract.
- **A hardcoded placeholder value that was never revisited:** a table entry's "maximum size" value was hardcoded to a number that turned out to match an unrelated, previously-deleted buffer's old cap, not the actual ANS-required value for that specific query. **General lesson:** audit hardcoded "should be right" constants against their actual, specific requirement — don't assume a plausible-looking number was derived correctly just because it's in the right ballpark.
- **A double-cell dispatch mechanism that only supported single-cell returns:** adding `MAX-D / MAX-UD` (double-cell `ENVIRONMENT?` queries) required a genuine dispatcher extension, not just a table entry, since the existing table-driven mechanism only ever pushed one return value. Resolved by adding a *second*, structurally distinct table and

lookup loop for double-cell entries, rather than reworking the already-tested single-cell path to handle both shapes — lower risk than a single mechanism trying to do both.

3. Testing and verification methodology used this phase

- **MAME-based manual testing, turn by turn:** every fix was applied to source, then the user manually re-assembled and tested on real MAME hardware, reporting exact `.s` stack-display output back. This caught several bugs that static/structural checking alone would have missed entirely (the double-negation carry-corruption bug in particular — static verification showed everything should work; only real execution against a *specific, real* negative value revealed the actual defect).
- **Hand-tracing before delivering, not just after a bug report:** for substantial or subtle logic (the 32-bit negation, the `PICK / ROLL` stack-effect verification later in the session), tracing the exact register/memory state instruction-by-instruction *before* declaring something correct caught real errors before they were ever shipped — cheaper than a full MAME round-trip, though not a substitute for one on anything genuinely novel.
- **Independent second-opinion checking, but verify the checker too:** a quick independent simulation (in this case, Python modeling the stack operations) caught a genuine question about one result — but the *simulation itself* had a bug in how it modeled a two-step push sequence, which needed fixing before it could actually confirm anything. Lesson: a cross-check is only as good as its own correctness — don't treat "a second method agrees" as automatically more trustworthy without checking why they agree.
- **Structural verification on every change, regardless of size:** every source edit was followed by (a) a duplicate-symbol check across every conditional-assembly flag combination, (b) a dictionary/symbol chain walk confirming nothing was broken, (c) byte-exact reassembly verification for any split/multi-file source layout. Cheap, fast, catches an entire class of careless mistakes (typos, copy-paste collisions) before they ever reach real hardware testing.
- **A concrete instance of catching your own mistake before delivery:** a documentation edit landed in the wrong location because a text search matched an unintended *duplicate* occurrence of the same string elsewhere in the document (a Table of Contents entry, not the actual target heading). Caught by rendering the actual output and looking at it, not by trusting a text-extraction check alone — the text was technically present "somewhere," which passed a naive check, but was visibly wrong when actually rendered. **Lesson: verify by rendering the real, final artifact, not just by grepping for presence of expected text.**

4. Documentation-maintenance lessons (process, not content)

- **Generated documentation silently goes stale if not explicitly resynced after every source change.** Over many turns of active bug-fixing, a Glossary and an “Assembler Source” appendix (embedded, human-readable copies of the real source) both stopped being updated partway through — verified by direct comparison against the real source, not assumed. One glossary entry had drifted from merely *stale* to actively *wrong*: it explicitly claimed several features were “missing” that had since been implemented and confirmed working. **Lesson: if you maintain a human-readable mirror of your source (glossary, appendix, changelog) for any reason, treat “resync the mirror” as part of the definition-of-done for every source change, not a separate, later cleanup task** — or at minimum, periodically audit it against the real source rather than assuming it’s still accurate.
- **A checklist/changelog file that’s copied into multiple places (a full version and a condensed version embedded in a formatted document) can drift between the copies even when the full version itself is current.** Found and fixed several places where the full, authoritative checklist had already resolved an item, but a condensed copy embedded elsewhere still showed it as open — a pure sync gap, not a content gap. If you maintain a “full” and “condensed” copy of the same information, decide explicitly which is authoritative and build in a habit (or better, an automated check) for keeping the other one in sync.
- **When asked “does X still need action,” actually check rather than trust memory of what was likely fixed.** Several supposedly-open items turned out to already be resolved (confirmed via direct inspection of the real, current source) — the checklist just hadn’t been updated to reflect it. Conversely, other items that *looked* similar on the surface (address/overlap questions) needed genuinely different treatment than expected once actually investigated — e.g. one “duplication” concern turned out to be a real, permanent architectural constraint (a fixed-width header field) misattributed to a different, unrelated cause (an already-fixed buffer redesign) in the original write-up. Don’t assume two superficially similar-sounding open items need the same fix without checking each independently.

5. Design patterns worth carrying to a fresh implementation

- **A shared “transient region,” floating relative to the current compile pointer, serving two opposite-growing purposes.** This system’s `WORD` buffer and pictured-numeric-output (`HOLD`) buffer share one fixed-size gap between the current compile

pointer and `PAD`, growing toward each other from opposite ends, sized so they provably can't collide even in the worst case (traced by hand: the combined worst-case usage of both exactly touches the shared boundary with zero overlap, by construction of the sizing constants). This is a reusable memory-layout pattern for any resource-constrained Forth implementation, independent of CPU.

- **An assembly-level unit test framework, gated by a conditional- assembly flag, that reverts to pure padding when excluded** — so enabling/disabling tests doesn't change the ROM's overall byte budget unpredictably. Each test independently saves and unconditionally restores the data stack pointer around itself, so a failed assertion mid-test can never leak stack residue into the next test. A shared reporting routine (print test name, print pass/fail, newline) avoids duplicating that logic per test. This pattern generalizes directly to any target.
 - **Computing derived documentation values (e.g. "how many cells does the return stack hold") from the real boundary constants rather than hardcoding the number** — so the documentation self-corrects through future memory-map changes instead of silently going stale. Small extra effort at write time, meaningfully more durable.
 - **Real assembler output (a full listing file with computed addresses, fill directives actually executing, real measured sizes) is qualitatively more trustworthy than any manual estimate or static analysis** — several previously-uncertain facts (whether two regions were truly contiguous with zero gap, whether a padding directive genuinely worked, the real byte size of a large code region) were only conclusively settled once a real listing became available. Manual/static estimates in this session were consistently close but not exact (one was off by about 4%) — treat them as provisional until a real toolchain confirms them, and say so explicitly in any documentation that relies on them in the meantime.
-

6. What this summary deliberately does not cover

- The specific 6809 opcodes, addressing modes, or exact register layout — none of that transfers to a 68008 implementation directly.
- The specific memory-map addresses — the *pattern* (shared transient region, computed rather than hardcoded derived constants) is what's reusable, not the numbers.
- The assembler-specific syntax (LWASM directives, conditional-assembly `IFEQ` / `ENDC` conventions) — check your own toolchain's equivalent features rather than assuming syntax compatibility.
- Work done *after* this summary's cutoff (new assembly-level unit tests covering the

stack-manipulation word set) — that's a separate, later phase not included here, per the requested scope.